Computer Networks 268 (2025) 111341

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

RLFE-IDS: A framework of Intrusion Detection System based on Retrieval Augmented Generation and Large Language Model

Xuewei Li [a,b,c], Zengyang Zheng [a,b,c], Mankun Zhao [a,b,c], Yue Zhao [a,d], Lifeng Shi [e], Baoliang Wang [a,d,*]

[a] College of Intelligence and Computing, Tianjin University, Tianjin, 300350, China
[b] Tianjin Key Laboratory of Cognitive Computing and Application, Tianjin, 300350, China
[c] Tianjin Key Laboratory of Advanced Networking, Tianjin, 300350, China
[d] Information & Network Center, Tianjin University, Tianjin, 300350, China
[e] DBAPPSecurity Ltd., Hangzhou, 310052, China

ARTICLE INFO

Keywords:
Intrusion detection
Large Language Model
Retrieval-Augmented Generation

ABSTRACT

Intrusion Detection Systems (IDS) play a critical role in network security as a key defense measure, often struggle to effectively handle unknown attacks or variations of known attacks. This challenge is exacerbated by the poor generalization of deep learning models. To enhance the adaptability of IDS, this article introduces an innovative framework called LLM-IDS, which explores the feasibility of leveraging Large Language Model (LLMs) for intrusion detection, due to its strong generalization capabilities. However, there is a significant difficulty in deploying LLMs. Moreover, since most LLMs are primarily designed for Natural Language Processing (NLP) tasks, significant differences arise when naively adapting them to intrusion detection tasks. To address them, this article introduces a novel framework called RLFE-IDS, comprising two key modules: Retrieval-Augmented Generation (RAG) and an embedding model called FE-Net. RAG employs a vector database to store network data alongside their corresponding vector representations. Based on the RAG framework, LLMs can be directly called through an Application Programming Interface (API), alleviating the difficulties in its deployment. The embedding model FE-Net, bridges the semantic gap between text data and network data. Upon receiving new network data, RLFE-IDS employs RAG to query the database for the most relevant network data, which is then fed into the LLM to classify. This article validates approach through experiments on four datasets, and deploys RLFE-IDS into the real network environment. Experiments show that the optimal accuracy of LLM-IDS is 99.36%, and that of RLFE-Net is 98.56%. The results demonstrate not only the feasibility of applying LLMs to intrusion detection, but also the robustness and superior performance of RLFE-IDS.

1. Introduction

With the rapid advancement of internet technology, network security challenges have become increasingly critical. Intrusion Detection Systems (IDS), as a key network defense mechanism, plays an important role, is the cornerstone of network security. It can monitor intrusions in real time, effectively protect data and systems, and support the stable operation of various fields [1].

The detection methods used by IDS are primarily divided into two categories: signature-based methods and anomaly-based methods. The signature-based methods identify network attacks by matching data against a database of known signature features. The anomaly-based methods detect attacks by analyzing network traffic behavior and identifying deviations from normal patterns [2]. While signature-based methods rely heavily on the construction of a comprehensive signature feature database, the efficacy is limited when dealing with unknown or novel attacks. This limitation often results in lower detection rates and higher false alarm rates. Anomaly-based methods, which are more flexible and scalable, are capable of dealing with some simple network attack scenarios [3].

In recent years, the network attacks have become more frequent and complex. This has led to traditional intrusion detection systems consuming a large amount of human and material resources during detection, resulting in longer processing times and making them struggle to cope. In order to solve this problem, researchers have further developed

* Corresponding author.
E-mail addresses: lixuewei@tju.edu.cn (X. Li), 2022244203@tju.edu.cn (Z. Zheng), zmk@tju.edu.cn (M. Zhao), zhao_yue@tju.edu.cn (Y. Zhao), 18610688855@163.com (L. Shi), wbl@tju.edu.cn (B. Wang).

https://doi.org/10.1016/j.comnet.2025.111341
Received 27 December 2024; Received in revised form 14 March 2025; Accepted 24 April 2025
Available online 22 May 2025
1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

machine learning-based and deep learning-based methods to enhance detection capabilities and address evolving network threats [4].

Machine learning-based methods use machine learning models to monitor and detect network traffic in real time. Xinghua Li et al. (2021) [5] proposed an Ensemble Learning method, which considered the varying sensitivity of different machine learning techniques to the data. This approach achieved a detection rate of 98.07% on the NSL-KDD dataset.

Deep learning-based methods leverage deep learning models to detect network traffic. These models provide stronger feature learning and generalization capabilities compared to traditional machine learning models. Jinghong Lan et al. (2022) [6] proposed an intrusion detection system that combined the Convolutional Neural Network (CNN), the AutoEncoder (AE), and the Prototype Network, with the addition of an attention mechanism. This model achieved an accuracy rate of 99.42% on the CIC-IDS2017 dataset.

Despite the advancements in the use of machine learning and deep learning models, the defense capabilities of IDS have been enhanced. However, there is still a challenge remaining in real-world scenarios. When detecting unknown attacks or variants of known attacks, due to the scarcity or even complete absence of data, both machine learning models and deep learning models lack the ability to precisely extract the features inherent in the data, machine learning-based and deep learning-based IDS may have a decline in detection rates [7].

To address this issue, researchers have started to focus on how to improve the generalization ability of intrusion detection systems. Thanks to the development of deep learning technology, few-shot learning has gradually caught the attention of researchers. Few-shot learning enables intrusion detection systems to learn from a small number of labeled samples, thus acquiring the ability to detect new types of attacks. Congyuan Xu et al. (2020) [8] applied few-shot learning to IDS for the first time, leading to the proposal of FC-Net. With only five labeled samples of new data for training, the model achieved an accuracy rate of 94.13% on the CIC-IDS2017 dataset.

In recent years, Large Language Models (LLMs) have demonstrated strong few-shot learning capabilities, which have attracted the attention of researchers. Although LLMs were originally designed to process and generate natural language text for solving Natural Language Processing (NLP) tasks, LLMs possess powerful generalization abilities. With fine-tuning, LLMs can perform exceptionally well in a wide range of fields, including healthcare, education, law, and scientific research [9].

However, there are the following challenges in directly applying LLMs to detect network traffic data:

1. For IDS, it is important to be able to achieve practical deployment and application, but there is a significant difficulty deploying and fine-tuning LLMs in real-world scenarios. LLMs usually contain billions of parameters, posing significant challenges for deployment. The number of parameters in existing LLMs is on the order of billions [10]. Whether for simple inference or fine-tuning training, a substantial amount of computational resources is required. Additionally, fine-tuning not only demands high computational power but also often requires a large volume of high-quality data, further increasing the resource requirements.

2. Intrusion detection systems need to recognize the dependency relationships among the features of network data. However, without fine-tuning, LLMs are unable to effectively extract and identify the features of network data. LLMs are designed to solve NLP problems, where the input is typically text data. This presents a challenge when adapting LLMs for network security, as there are structural differences between network traffic data and text data. Text data is unstructured and rich in semantics, it cannot be organized in a specific form [11], while network traffic data is structured and has weaker semantic meaning, it can be organized in a specific form [12].

To address the challenges mentioned above, RLFE-IDS framework is proposed in this article which comprises two main components: RAG and FE-Net. RAG, full name Retrieval-Augmented Generation, obtains relevant knowledge by retrieving from the knowledge database and incorporates it into the prompt, enabling LLMs to refer to the corresponding knowledge and thus give reasonable answers. It allows direct use of LLMs through API calls, enabling users to avoid the need to deploy LLMs on their own machines. This reduces both the deployment complexity and the computational resource requirements. Additionally, to bridge the gap between differences between network traffic data and text data, FE-Net is proposed, a custom embedding model for network traffic data. Unlike relying on open-source embedding models, FE-Net converts network traffic data into feature vectors and stores them in the knowledge database. This enhances the efficiency of the knowledge database, and preserves the original features of the data, enabling the retrieval of the most relevant knowledge based on specific queries. Above all, the innovations and contributions of this article are as follows:

1. This article proposes LLM-IDS framework, introduces LLMs into the field of intrusion detection, and demonstrates the feasibility of using fine-tuned LLMs to detect network traffic data.

2. This article introduces RLFE-IDS framework, a deep learning framework built on the RAG architecture. By retrieving information from knowledge database and directly calling the API of LLMs, this framework reduces the deployment complexity of LLMs.

3. This article proposes FE-Net, a embedding model designed to address the differences between network traffic data and text data. This model transforms network traffic data into feature vectors and stores them in a vector knowledge database, preserving the original data features.

4. This article conducts experiments using four benchmark datasets and real-world network traffic data, demonstrates the superiority and robustness of the proposed model, which outperforms other advanced models in terms of accuracy.

The work of this article will be presented in the following form: Section 2 will mainly introduce the existing deep learning techniques and LLMs in the IDS field. Section 3 will mainly introduce the framework of RLFE-IDS and FE-Net. Section 4 will mainly introduce the model experiments and the evaluation of RLFE-IDS. Section 5 will provide a summary of this entire article.

2. Related work

In this section, this article introduces some machine learning, deep learning and few-shot learning models used in the field of intrusion detection. After that, Large Language Models are introduced.

2.1. IDS with machine learning

Intrusion detection systems are one of the important means to help people resist network attacks in real life. With the increasing complexity of network data, intrusion detection systems based on machine learning have become more popular. Emmanuel Chinanu Uwazie et al. (2024) [13] compared the classification effects of the K-Nearest Neighbor (KNN), Random Forest (RF), and Support Vector Machine (SVM) on different datasets. By adjusting the parameters, each algorithm achieved different accuracies on various datasets. Among them, the KNN algorithm had the highest accuracy rate on all datasets, with accuracy rates of 99.99%, 99.69%, and 97.16% on the NSL-KDD, CIC-IDS2017, and CSE-CIC-IDS2018 datasets respectively. Chuanpu Fu et al. (2021) [14] proposed a real-time malicious traffic detection system named Whisper. This system encodes the packet feature sequences into vectors, extracts the features in the frequency domain, and finally uses the K-Means algorithm to conduct clustering judgments on the frequency domain features. The system achieved an accuracy rate of 99% in an environment with low latency and high throughput. In order to extract more effective features without increasing the data processing time, Ahmet Nusret Özalp et al. (2022) [15] proposed

an intrusion detection system for detecting network attacks based on high-frequency features. This method used multiple attribute selection methods to determine the high-frequency features, employed various machine learning algorithms to classify the feature vectors and evaluate the performance. Eventually, this method found that the Random Forest algorithm had the highest classification accuracy rate, reaching 99.76%. Thi-Thu-Huong Le et al. (2022) [16] proposed an intrusion detection system based on an ensemble of trees (Decision Tree and Random Forest) and the SHapley Additive exPlanations (SHAP) method. This method used information gain to construct decision trees, employed the Bootstrap Aggregating (Bagging) algorithm to build random forests. Finally, this method used the SHAP method to explain the influence of features on the output and the reasons for the model's decisions, improving the interpretability of the system and achieving an average accuracy rate of 99%. Victor Chang et al. (2023) [17]presents a novel framework for enhancing intrusion detection systems by classifying network traffic into normal and attack categories. By integrating multiple machine learning algorithms, the proposed method achieved a detection accuracy of over 98% when evaluated on the KDDCUP-1999 dataset.

2.2. IDS with deep learning

As network data becomes increasingly complex, IDS based on deep learning models have gained popularity. Wei Wang et al. (2017) [18] were the first to convert network data into image data and apply representation learning to network data classification. This method defined a "Flow" as packets with the same source IP, source port, destination IP, destination port, and protocol. The first 784 bytes from each Flow were extracted to form a 28×28 image, which was then classified using a CNN. This method achieved an average accuracy of 99.41%, but it had poor generalization. To enhance the generalization ability of the model, Jinghong Lan et al. (2022) [6] proposed an IDS with the MEMBER architecture, consisting of three main components: MAE, MAC, and DP. The MAE extracts statistical features from the original network data, the MAC extracts packet-content features, and the DP calculates the geometric distance between the feature vector and the prototype. MEMBER addresses the performance degradation caused by unbalanced network data and achieves 99.42% on benchmark dataset. However, this complex model increases the detection time. To reduce the time complexity, Jung-San Lee et al. (2022) [19] proposed an IDS based on Deep Neural Networks (DNN) and AE, introducing an agile training mechanism that updates only the weights with significant impact on the model's results. This approach reduced training time by 30%. Laisen Nie et al. (2022) [20] found that the multi-classification accuracy of the existing models is relatively low, so a separate Generative Adversarial Network (GAN) was trained for each type of data, then combined the discriminators of all the GANs and classified the data. This model achieved a precision rate of 99.59% and a recall rate of 98.76%. However, IDS based on deep learning models require a large amount of training data. When there is insufficient training data, the accuracy will decline.

2.3. IDS with few-shot learning

To address the growing frequency of unknown network attacks in society, many researchers have proposed IDS based on meta-learning and few-shot learning. Meta-learning, also known as "learning to learn" [21], is a method for improving a model's ability to learn from few examples. Few-shot learning is a specific application of meta-learning in supervised learning, aiming to build accurate models using a small number of training examples. Congyuan Xu et al. (2020) [8] were the first to apply few-shot learning to the IDS field, proposing FC-Net. FC-Net consists of two components: a feature extraction model called F-Net, and a model called C-Net, which calculates the distance between feature vectors. FC-Net achieved an accuracy rate of approximately 94%, and could detect new types of network traffic samples with only a small number of labeled samples. Jingcheng Yang et al. (2022) [22] found that F-Net ignores statistic features of network data, so FS-IDS was proposed based on few-shot learning, which uses AE to extract statistical features from network traffic and CNN to extract feature vectors and compute the distances between them. Experimental results showed that with only five samples, the model achieved an accuracy rate of 97%. Chaomeng Lu et al. (2023) [23] introduced the Model-Agnostic Meta-Learning (MAML) framework in IDS, proposing an IDS based on CNN and MAML. MAML uses a meta-learner to train a base learner, allowing it to "learn how to learn". This approach, using five benchmark datasets for training, achieved an accuracy rate of 90% with just five samples. Zhixin Shi et al. (2023) [24] further improved MAML by introducing the L2F forgetting algorithm. This algorithm accelerates the model's adaptation to different tasks, enabling faster convergence. With the addition of the Learning to Forget (L2F) algorithm, the model's accuracy increased by nearly 4.5%, reaching 70% just after 10 training epochs.

2.4. Large language model

LLMs (Large Language Models) are based on deep learning and artificial intelligence techniques, typically referred to as Transformer-based models, containing hundreds of billions (or more) of parameters [10]. LLMs are designed for processing and generating natural language text to solve Natural Language Processing (NLP) tasks. These models exhibit remarkable learning capabilities and represent a significant advancement in the field of NLP. The release of GPT-3 [25] marked a milestone in the development of LLMs. Prior to GPT-3, LLMs like T5 [26], GPT-1 [27], and GPT-2 [28] had around 10 billion parameters. In contrast, GPT-3 increased the number of parameters to 175 billion and introduced the concept of in-context learning (ICL) [29]. ICL enables GPT-3 to excel at a wide variety of NLP tasks and also improves its performance in reasoning tasks [30]. Since then, experts have recognized the immense potential of LLMs beyond NLP, with applications emerging in various fields. LLMs are pre-trained on large, diverse text corpora, allowing them to acquire knowledge from multiple domains and thus function as domain experts. Nowadays, LLMs have been widely applied in many non-NLP fields. The LLM used in this article is the ChatGLM3-6B model, jointly released by Zhipu AI and KEG Lab at Tsinghua University [31]. ChatGLM3-6B delivers performance comparable to GPT-4V, while maintaining a relatively low parameter count. In addition, the RAG framework which is a application of LLMs is used. The core of RAG can is Retrieval and Generation. The former mainly takes advantage of vector databases to retrieve target knowledge, while the latter makes use of LLMs and prompt engineering, to reasonably utilize the retrieved knowledge and generate accurate answers.

3. Framework of RLFE-IDS

In this chapter, this article introduces RLFE-IDS, which is built on the RAG framework and FE-Net. First, the detection process of RLFE-IDS is described. Next, FE-Net is detailed designed for processing network traffic data. Following that, the process of retrieving the most relevant network traffic data is explained. Finally, how prompts are constructed based on the retrieved data is outlined, which enables LLM to analyze network traffic and detect potential threats using knowledge. The main framework of RLFE-IDS is illustrated in Fig. 1.

In Fig. 1, RLFE-IDS involves two processes: knowledge database and retrieval-augmented generation. Knowledge database is a vector database, which stores knowledge. This article textualizes the data and uses FE-Net to embed data, then combines the text and the vector, finally stores in the knowledge database. Retrieval-augmented generation is the processing procedure of the model when encountering new data. The new data is used to conduct retrievals in the knowledge database, and then transmit the retrieval results to the LLM. Finally, this article let the LLM make classification judgments.

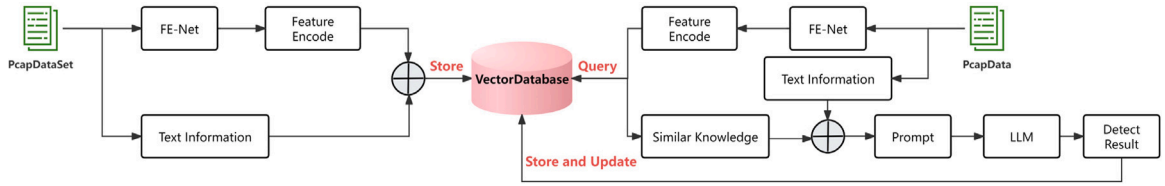


Fig. 1. The framework of RLFE-IDS.

3.1. Overview

As illustrated in Fig. 1, RLFE-IDS involves two main processes: knowledge database construction and retrieval-augmented generation.

The knowledge database is essentially a vector database that stores known information in vector form. To construct this knowledge database, this article first converts the raw network traffic data into tabular data using the CICFlowMeter tool [32]. CICFlowMeter is one of the greatest tools that can be used to generate bi-directional flows. It can convert PCAP files to CSV files by generating bidirectional flows [33]. After this conversion, the data undergoes two parallel processes: **Feature Vector Generation**: The tabular data is passed through the FE-Net, an embedding model, which transforms it into 1024-dimensional feature vectors that retain key information about the network traffic. **Text Data Generation**: Textual descriptions are created to record the names and values of each feature. These textual records correspond one-to-one with the generated feature vectors and are stored in the knowledge database.

Retrieval-augmented generation leverages LLM for detecting anomalies in new network traffic data. It is about retrieving relevant knowledge and integrating it into the prompt, allowing the large model to reference the corresponding knowledge to provide reasonable answers. When new network traffic data, denoted as n, is received: **Preprocessing**: The CICFlowMeter tool generates tabular data, which is then converted into a feature vector v using the FE-Net and text data t. **Data Retrieval**: The feature vector v is used to search the knowledge database, retrieving the most similar known network traffic data, denoted as $\tilde{t}$. **Prompt Construction**: The retrieved data $\tilde{t}$ is used as the context, and the text data t is treated as the query to construct a prompt. This prompt is then input into the LLM for detection. **Dynamic Knowledge Update**: LLM outputs are immediately persisted in the knowledge repository, fostering a closed-loop system where the model continuously benefits from newly acquired information without retraining.

3.2. Knowledge database construction

The knowledge database plays a crucial role in storing knowledge in vector form [34]. Its overall structure is depicted in Fig. 2.

In Fig. 2, the construction of knowledge database can be broken down into two main components: Embedding Model (FE-Net) and Textualization Method. The FE-Net model is designed to convert network traffic data into a 1024-dimensional feature vector. The textualization method is responsible for transforming tabular data into text data.

Embedding: This process transforms network traffic data into vector representations. These vectors serve as the core of the knowledge database, enabling efficient storage and retrieval of data.

Text Extraction: This step involves extracting and storing detailed textual information about the network traffic data. This textual information is linked to the corresponding vectors, allowing the knowledge database to map vectors back to the associated network traffic data accurately.

3.2.1. Feature extraction

The original network traffic data is represented as binary strings and is typically stored in files using the pcap format. However, interpreting these binary strings requires specialized knowledge, akin to deciphering a complex password. To make this data more accessible and meaningful, the CICFlowMeter tool is used to extract traffic features from the raw network traffic data. This process transforms binary strings into structured tabular data that contains packet feature information, as illustrated in Table 1. However, within these feature information, elements such as source IP addresses and destination IP addresses may contain user privacy-related content. Therefore, data anonymization is performed at the final stage to remove privacy-related features.

3.2.2. FE-Net

Given the structural differences between text data and network traffic data, this article opts not to use open-source embedding models. Instead, a dedicated embedding model named FE-Net is developed. Unlike text data, which is rich in semantics, network traffic data has low semantic content. Text data is a sequence of discrete symbols, and there is no specific way to organize it. Its semantics are rich and explicit, and information can be directly conveyed through grammar and vocabulary. However, it depends on the relationships between preceding and following words, and the influence of the context needs to be considered when embedding. On the other hand, network traffic data is more similar to image data. It is organized in the form of a CSV table, with meager and implicit semantics and little context dependence. When embedding, the correlation between features and the final label needs to be focused on. Same to the open-source embedding models, FE-Net is designed to map the data into vector, which can store the feature of data. However, the open-source embedding models are trained to capture the semantic information in text data, FE-Net is trained to capture the key feature information in network traffic data.

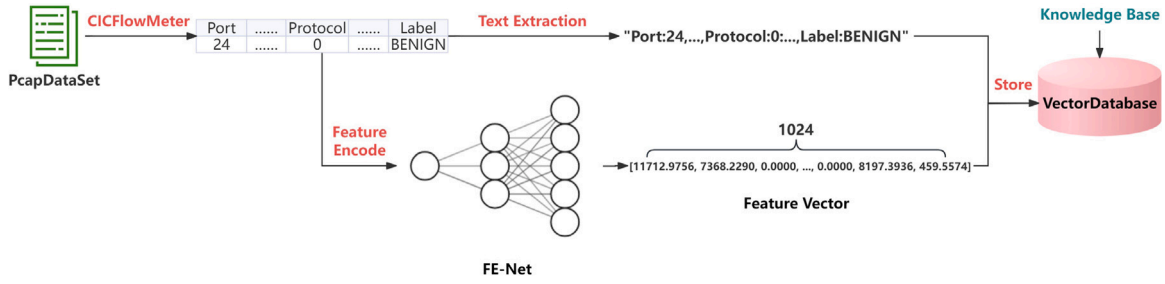


Fig. 2. The framework of Knowledge Database Construction.

Table 1
The tabluar data of network data after using CICFlowMeter.

Port	Flow Duration	… …	FIN Flag Count	SYN Flag Count	… …	Label
6	112740690	… …	0	1	… …	BENIGN
6	112740560	… …	0	1	… …	Bot
0	113757377	… …	0	0	… …	FTP-Patator
17	100 126	… …	0	0	… …	DoS Hulk
0	54 760	… …	0	0	… …	PortScan

FE-Net contains four convolution layers and two full connected layers. It is used to transform the network data into 1024-dimensional vector. Its input is 128-dimensional vector and its output is 1024-dimensional vector. The framework of FE-Net is illustrated in Fig. 3.

The input to FE-Net is a 128-dimensional vector. Since network traffic data may vary in dimension, this article unifies the data dimension to 128 before input. If the data dimension is insufficient, zeros are appended; if it exceeds the required size, the excess part is truncated. FE-Net consists of four convolutional layers and two fully connected layers. The convolutional layers use Conv1d with kernel sizes of 1, which can effectively capture abstract features and inter-feature dependencies from input data. Each convolutional layer is followed by a normalization layer and the ReLU activation function. After the final convolutional layer, an average pooling layer is applied to reduce dimensionality while preserving the core features of the data. The two fully connected layers are responsible for learning the feature representations of the data. The first fully connected layer maps input dimensions from 4096 to 2048, and the second layer reduces the dimensions further to 1024.

3.2.3. Training strategy of FE-Net

The task of FE-Net is to generate embeddings, whereas intrusion detection is fundamentally a classification task. Since these tasks differ, a classifier needs to be constructed during training. The Prototype Network [35] is adopted for this purpose. The core idea of Prototype Networks is to classify objects based on similarity by searching for and measuring data proximity, which aligns closely with the concept of querying the knowledge database. In the Prototype Network, class prototypes are constructed from the support set data. A class prototype represents the mean feature vector of all samples within a particular class, it is computed as follows:

$$p_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\phi(x_i) \tag{1}$$

where p_k represents the prototype vector of class k, S_k represents all samples of class k in the support set, f_ϕ is the feature extractor.

After calculating the prototypes, the query set data computes its similarity to all prototype vectors. The class with the highest similarity is then assigned as the predicted label for the query data. Compared to the standard Prototype Network, a simplified training strategy is employed. Instead of dividing the data into a support set and a query set, the strategy randomly selects a subset of samples from each class to construct prototypes. These prototypes are encoded by the FE-Net model into feature embeddings. The embedding process can be summarized as follows:

$$v_x = f_\phi(x) \tag{2}$$

where v_x represents the vector of the sample x, and f_ϕ is the FE-Net model. Then, the Euclidean distance is used to measure the similarity between the feature embeddings of the data and the class prototypes:

$$d(x, P) = \bigcup_{k=0}^{P_k} \sqrt{\sum_{x_i \in V_x, p_i \in P_k} (x_i - p_i)^2} \tag{3}$$

where P_k represents the prototype vector of kth class, x_i is the value of v_x, and p_i is the value of P_k. Throughout the training process, the Cross-Entropy is used as the loss function and the backpropagation algorithm to optimize the FE-Net model:

$$\mathcal{L}(f_\phi, P) = -\sum_{i=0}^{n} y_i log(d(x_i, P)) \tag{4}$$

where y_i is the ground truth of sample x_i, and n represents the number of training data. When a epoch of training is completed, every class prototype vector will be updated via Eq. (1).

3.2.4. Text extraction

To better store network traffic data and minimize information loss, each processed network traffic data will be converted into text. The

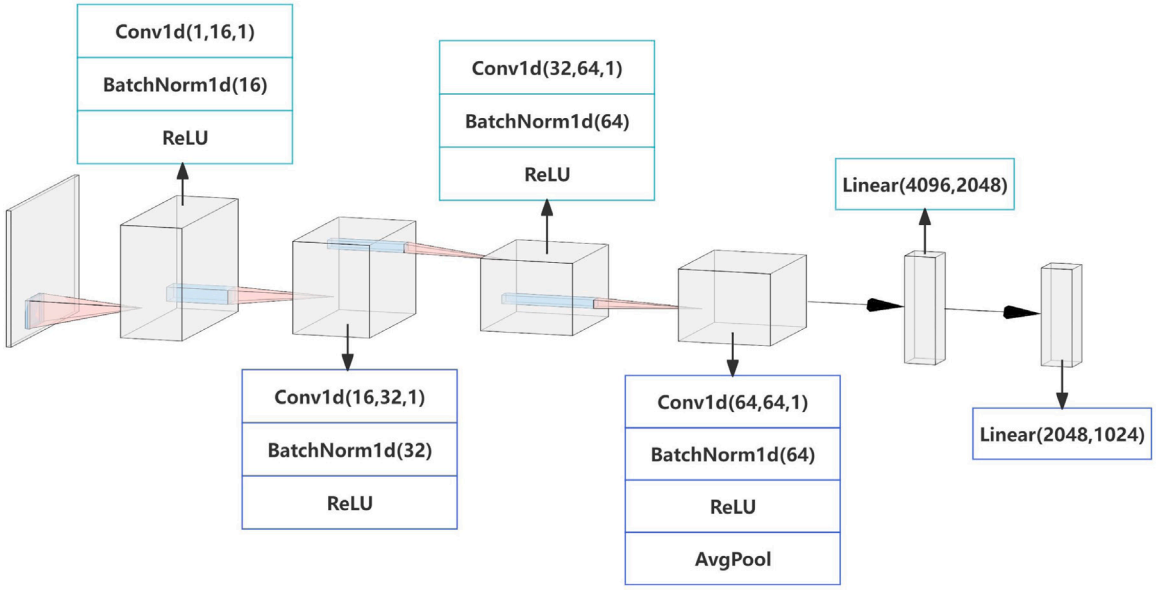


Fig. 3. The framework of FE-Net.

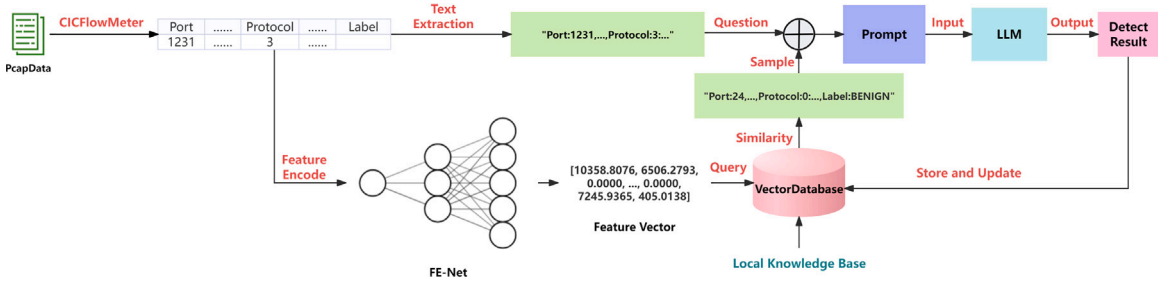


Fig. 4. The framework of Retrieval-Augmented Generation.

content of the text will record the names and values of each feature in the data. These texts are referred to as traffic text data. Each piece of traffic text data will correspond to a unique feature vector.

3.2.5. Store

After obtaining the traffic text data and its corresponding vector data, RLFE-IDS combines both and store them in the vector knowledge database. The traffic text data captures the semantic information of the network traffic, while the vector data represents the feature information. In the vector database, the system can efficiently locate the corresponding traffic text data by querying the vector data. When users wish to query similar network traffic data, the vector database analyzes the query vector, computes the similarity between the query vector and others stored in the database, and quickly retrieves the most relevant traffic text data. This enables the system to provide accurate results to users.

3.3. Retrieval-augmented generation

Retrieval-augmented generation is a key processing step in RLFE-IDS when handling new network traffic data. Its specific structure is illustrated in Fig. 4. It contains retrieval and augmented generation. Retrieval is to find the most similar data to the query data. Augmented generation is to prompt LLM based on the result of retrieval.

When new network traffic data arrives, it first undergoes embedding and text extraction. The resulting feature vector is then used to query the knowledge database. Finally, the retrieved results are provided as context to the LLM, enabling it to determine whether the network traffic data corresponds to an attack based on the provided context.

3.3.1. Retrieval

3.3.2. Augmented generation

Retrieval is the process in which, when new network traffic data arrives, the model queries the database to find the data most similar to it, and provides this data to the LLM [36]. Since FE-Net fully accounts for the correlation between features and the final label during the embedding process, two distance algorithms are proposed to measure similarity: the Euclidean distance and the Mahalanobis distance.

The Euclidean distance is a commonly distance definition, which represents the actual distance between two points in an m-dimensional space. The Euclidean distance calculated as follow:

$$d_E(x,V) = \bigcup_{j=0}^{V_j} \sqrt{\sum_{x_i \in x, v_i \in V_j} (x_i - v_i)^2} \tag{5}$$

where V_j is the jth vector in the knowledge database, x_i is the value of x, and v_i is the value of V_k. This algorithm is simple and intuitive, can reduce the complexity of the model.

The Mahalanobis distance is a distance measurement method used to measure the distance between a sample point and a distribution. It takes into account the covariance structure of the data and is calculated based on the mean and covariance of the sample data. The Mahalanobis distance calculated as follow:

$$d_M(x,V) = \bigcup_{j=0}^{V_j} \sqrt{(\boldsymbol{x} - \boldsymbol{V}_j)^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{V}_j)} \tag{6}$$

where V_j is the jth vector in the knowledge database, Σ^{-1} represents covariance matrix of variables. This algorithm eliminates the influence of correlation and scales differences between variables, can reflect the true similarity or difference between data.

Augmented Generation builds upon retrieval by using the results as context for prompting the LLM. Ben Mann et al. (2020) [25] explained why LLMs can perform few-shot or even zero-shot learning. By providing high-quality samples, LLM is allowed to understand human intention and expectation through the given context, thus enabling it to generate the desired results [25]. In this process, prompt engineering is utilized to embed the retrieval results into a prompt template. This enables the generation of exclusive prompt for each network traffic data, which are then input into the LLM to generate the result. In-context learning(ICL) is used to form the prompt. In-context learning is a paradigm that allows language models to learn tasks given only a few examples in the form of demonstration [37]. The core of ICL is to make use of the ability of LLMs to understand the context, and enable LLMs to generate answers based on the given context information. The prompt constructed by using ICL usually includes the task description, the role that the LLMs need to play, the given samples, and the problem description. The specific format of the prompt is shown in Table 2.

In Table 2, "Role" column represents the role LLM needs to play. "Samples" column represents the results which are retrieved from the database. "Question" column represents the input data. The three

Table 2
The format of the prompts.

Input			Output
Role	Samples	Question	
You are the Intrusion Detection System.	-Input: DestinationPort:5463.0, FlowDuration:3.0, … … , IdleMax:0.0, IdleMin:0.0, -The label: BENIGN	There is a network data, please prdict its label: DestinationPort:40462.0, FlowDuration:3.0, … … , IdleMax:0.0, IdleMin:0.0,	BENIGN

Table 3
The class label of three dataset.

Dataset	Label
CIC-IDS2017	Benign, Bot, DDoS, DoS slowloris, DoS Hulk, DoS GoldenEye, DoS Slowhttptest, PortScan, Web Attack-Sql Injection, Web Attack-Brute Force, Web Attack-XSS, Heartbleed, Infiltration, FTP Patator, SSH Patator
CSE-CIC-IDS2018	Benign, SQL Injection, Bot, Brute Force -Web, Brute Force -XSS, SSH Bruteforce, FTP BruteForce, DDoS attack-LOIC-HTTP, DDoS attack-HOIC,DDoS attack-LOIC-UDP, DoS attacks-SlowHTTPTest, DoS attacks-GoldenEye, DoS attacks-Slowloris, Infilteration
CIC-DDoS2019	Benign, NetBIOS, TFTP, UDP, Portmap, DNS, UDPLag, WebDDoS, MSSQL, NTP, SNMP, SSDP, Syn, LDAP
CICIoT2023	Benign, DDoS-UDP_Flood, Recon-HostDiscovery, DDoS-SlowLoris, DDoS-ICMP_Fragmentation, MITM-ArpSpoofing, SqlInjection, Mirai-greip_flood, Recon-PortScan, DDoS-RSTFINFlood, DoS-TCP_Flood, DoS-UDP_Flood, DDoS-SYN_Flood, Mirai-udpplain, VulnerabilityScan, DNS_Spoofing, DoS-SYN_Flood, BrowserHijacking, Recon-OSScan, DDoS-TCP_Flood, Backdoor_Malware, DoS-HTTP_Flood, DDoS-ACK_Fragmentation, DictionaryBruteForce, Uploading_Attack, DDoS-UDP_Fragmentation, DDoS-ICMP_Flood, Recon-PingSweep, Mirai-greeth_flood, CommandInjection, XSS, DDoS-HTTP_Flood, DDoS-PSHACK_Flood, DDoS-SynonymousIP_Flood

columns of "Role", "Samples" and "Question" are combined to form a sentence, which serves as the input of LLM. Finally "Output" column represents the output of LLM.

Finally, upon detection of a network data instance by the LLM, the extracted text features (the result of Text Extraction), the feature vector (the result of FE-Net), and final label (the output of LLM) are aggregated into a new object stored within the knowledge database. By doing so, information about newly detected attacks is immediately incorporated into the repository, enabling real-time knowledge augmentation for the LLM and avoiding the need for retraining.

4. Experiments

In this chapter, this article introduces the experimental setup and details, four commonly datasets are used to test and analyze RLFE-IDS. Additionally, a comparative analysis between RLFE-IDS and several representative models is performed.

4.1. Datasets

The datasets used in experiments are CIC-IDS2017, CSE-CIC-IDS2018, CIC-DDoS2019 and CICIoT2023. These datasets were created by the Canadian Institute for Cybersecurity (CIC) to simulate real-world attack scenarios. These four datasets are widely adopted in the intrusion detection field and are highly recognized for the established reputation in benchmarking security systems. In addition, these four datasets are highly credible datasets containing real network traffic and multi-class attacks, widely adopted for intrusion detection evaluations due to the comprehensive attack coverage and realistic operational scenarios.

The CIC-IDS2017 dataset collected traffic data over a period of 5 days in 2017, and it includes attack types such as DoS, DDoS, and Heartbleed, among others. The CSE-CIC-IDS2018, a collaboration between CSE and CIC, gathered traffic data over 10 days in 2018, with attack types including DDoS, Brute Force, Bot, and others. The CIC-DDoS2019 specifically focused on various DDoS attacks and categorized these attacks into different classes. The CICIoT2023 offered internet of things (IoT) traffic profiles and diverse attack vectors, used a wide range of topologies and multiple devices and contained various attacks that do not appear in other datasets, providing a new resource for IoT security analysis. The specific classes of network traffic data collected by each dataset are detailed in Table 3.

In addition, RLFE-IDS is tested in the real network environment. In order to reduce the API calls of the RLFE-IDS, the form of segmented testing is adopted, which means this article collects network traffic data for a certain period of time such as 1 min, and then pre-processes these data centrally. After pre-processing operations, such as data anonymization and removal of missing values, the data is input into RLFE-IDS. This article collects network traffic data in a real network environment for 48 h. Finally, about 1,000,000 real network traffic data is collected, and the detail information is shown in Fig. 5.

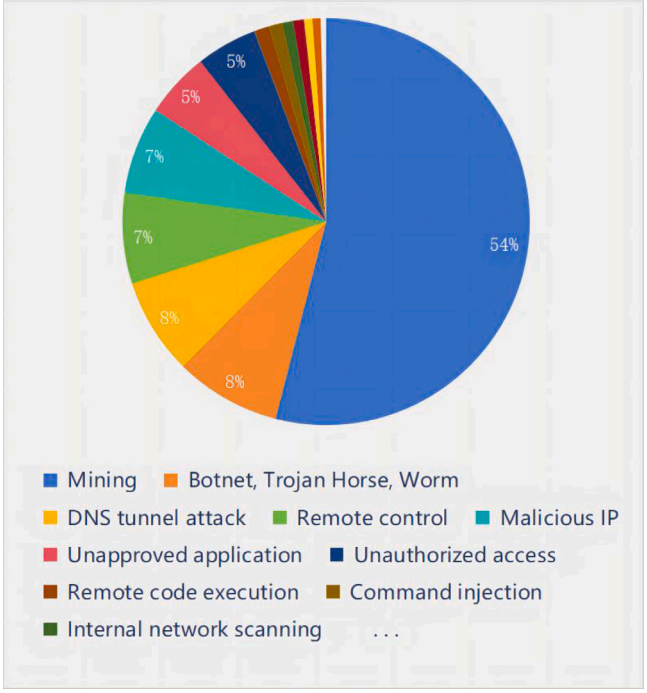


Fig. 5. The detail information of collected real network traffic data.

4.2. Experiment setting

The experiment consists of four main parts: proving the feasibility of using LLM for intrusion detection, training the FE-Net model, constructing the knowledge database, and performing retrieval-augmented generation.

LLM-IDS: Since LLMs were originally designed to solve NLP problems and intrusion detection is a network data classification task, directly applying an LLM to this problem is not feasible. To address this, RLFE-IDS fine-tuned the ChatGLM3-6B model using the LoRA method, leveraging 80% of the CIC-IDS2017 dataset for training. The LoRA method freezes the original parameters of the LLM and only retrains some parameters using the fine-tuning data, simplifying the training process. After fine-tuning, the trained parameters are injected into the LLM.

FE-Net Training: FE-Net was trained using the CIC-IDS2017 dataset with PyTorch, optimized using the SGD function. Only 80% of the CIC-IDS2017 dataset was used for training FE-Net, with the remaining 20% reserved for testing. Training was performed using SGD with a learning rate of 1e-3, batch size of 32, and 150 epochs.

Knowledge Database Construction: For the knowledge database, a vector database is used. Initially, each category contained 100 samples, and the database was dynamically updated as new data arrived. If the number of samples in a category fell below 100, all available samples in that category were stored.

Retrieval-Augmented Generation: During retrieval, RLFE-IDS respectively selected the 1, 3 and 5 most similar samples to the query data from the knowledge database, forming 1-shot, 3-shot and 5-shot few-shot learning tasks. These retrieved samples were then used as context for the LLM to perform intrusion detection.

All experiments are performed on Intel(R) Xeon(R) Gold 6348 CPU @ 2.60 GHz. LLM-IDS is trained and tested with 8 NVIDIA Graphics Device A800 GPUs and RLFE-IDS is trained and tested with 1 NVIDIA Graphics Device A800 GPU.

4.3. Comparison models

To prove the feasibility of LLM for intrusion detection, firstly, we compare LLM-IDS with two traditional machine learning models and four state-of-the-art deep learning models:

Support Vector Machine and Random Forest represent the most classical machine learning methods. DST-IDS is a comprehensive intrusion detection framework. SICNN and CL-SKD both exemplify state-of-the-art advancements in IDS. BT-TPF shares foundational approaches with LLM-IDS.

Support Vector Machine [38]: Support Vector Machine (SVM) is supervised learning model used for classification and regression tasks. SVM aims to find an optimal hyperplane that maximizes the margin between different classes in high-dimensional feature spaces.

Random Forest [38]: Random Forest (RF) is a supervised machine learning method. It operates based on the principle of ensemble learning, in which multiple decision trees are grown instead of a single decision tree to solve complex problems and improve the performance of the model.

DST-IDS (2022) [39]: DST-IDS is an integrated framework combining packet-based and flow-based IDS, utilizing Dempster-Shafer Theory (DST) to fuse results from both IDS. This hybrid approach provides robust detection by leveraging different layers of network traffic data.

SICNN (2024) [40]: SICNN optimizes the synaptic structure of CNN using the Synaptic Intelligence algorithm. This approach addresses the class imbalance issue in intrusion detection by introducing a specialized loss function that prevents the CNN model from forgetting past tasks.

CL-SKD (2024) [41]: CL-SKD proposes a two-stage lightweight intrusion detection framework based on self-supervised contrastive learning and self-knowledge distillation. It aims to reduce the reliance on labeled data, which can be expensive and time-consuming to acquire, and enhance the model's generalization ability.

BT-TPF (2024) [42]: BT-TPF model introduces knowledge distillation using a Vision Transformer (ViT) [43] as the teacher model to guide a Poolformer model. This framework uses the trained Poolformer model as the classifier for intrusion detection.

Table 4
The comparison between Euclidean distance and Mahalanobis distance.

Distance	Accuracy	Recall
Euclidean distance	98.02%	99.23%
Mahalanobis distance	93.32%	87.13%

Then, to prove the effectiveness of RLFE-IDS for intrusion detection, RLFE-IDS is compared with four state-of-the-art few-shot learning models:

FC-Net is the first method applying few-shot learning to IDS. FS-IDS improves upon FC-Net. MAML adapts the classical meta-learning approach to IDS. BFS-NID represents one of the latest advancements in IDS.

FC-Net (2020) [8]: FC-Net is a few-shot learning framework that includes two components: F-Net for feature extraction and C-Net for classification. The framework is designed to work effectively with a small number of labeled samples.

FS-IDS (2022) [22]: FS-IDS is an intrusion detection system that incorporates flow data encoding, feature fusion, and a few-shot learning framework. The system is designed to learn generalized knowledge about network traffic behavior, making it adaptable to new and unseen attacks.

MAML (2023) [23]: Model-Agnostic Meta-Learning (MAML) approach is a meta-learning model where the training process involves two layers: a meta-learning layer for model adaptation and a base-learning layer for task-specific learning.

BFS-NID (2024) [44]: BFS-NID is a few-shot class-incremental learning method specifically designed for network intrusion detection, using a branch fusion strategy to continuously learn new attack classes with minimal samples.

These comparisons are made to highlight the advantages and performance of the RLFE-IDS framework.

4.4. Comparison experiment

First, in order to make a comprehensive comparison between the Euclidean distance and the Mahalanobis distance in Section 3.3.1, these tow methods are implemented in RLFE-IDS for 5-shot task to determine which is the best architecture for measuring distance. The result is shown is Table 4. The result shown in Table 4 lists the accuracy and recall of two different distance in RLFE-IDS. The Euclidean distance shows the best performance in RLFE-IDS. Besides, the computational complexity of Mahalanobis distance—arising from covariance matrix estimation—scales cubically with feature dimensions, making it infeasible for processing of high-dimensional datasets compared to the linear complexity of Euclidean distance. Therefore, the Euclidean distance will be used in the subsequent experiments.

Then, feasibility comparison of the LLM experiment in intrusion detection is implemented. The detailed results are in Table 4. The comparison is made among LLM-IDS which do not use RAG, two traditional machine learning methods and four state-of-art deep learning models, with a special focus on BT-TPF, a model based on the ViT. The ViT model shares many technical concepts and architectural features with LLMs [43], making it a relevant comparison for assessing the potential of LLMs in network intrusion detection. As shown in Table 5, both of the two machine learning methods, Support Vector Machine (SVM) and Random Forest (RF), show excellent performance, which demonstrates that the machine learning framework is adaptable and has great potential in the field of intrusion detection. However, the results of these two models still have some gaps compared with the results of the other four deep learning models. Besides, BT-TPF outperforms all other models, including LLM-IDS. This model achieves the highest scores across all evaluation metrics, demonstrating that the deep learning framework has excellent stability and robustness for intrusion detection tasks.

Table 5
The comparison result in terms of Accuracy, Recall, Precision and F1-score for LLM and other models.

Models	Accuracy	Recall	Precision	F1-Score
SVM	98.18%	95.63%	98.74%	97.16%
RF	96.76%	97.62%	97.96%	97.78%
DST-IDS (2022)	99.38%	95.66%	98.60%	97.11%
SICNN (2024)	98.63%	98.63%	98.66%	98.62%
CL-SKD (2024)	99.33%	99.33%	97.38%	97.34%
BT-TPF (2024)	**99.60%**	99.60%	**99.60%**	**99.60%**
LLM-IDS (Ours)	99.36%	**99.67%**	99.53%	**99.60%**

Table 6
The comparison result in terms of Accuracy in 5-shot task for RLFE-IDS and other models.

Models	Accuracy 5-shot
FC-Net (2020)	94.13%
FS-IDS (2022)	97.51%
MAML (2023)	92.19%
BFS-NID (2024)	93.17%
RLFE-IDS (Ours)	**98.02%**

LLM-IDS, also performs excellently. Although its Accuracy (99.36%) is slightly lower than that of BT-TPF, its Recall (99.67%) and Precision (99.53%) are close to or even exceeded other methods, and its F1-score (99.60%) is the highest alongside BT-TPF, which reflects a good balance between Precision and Recall. Importantly, LLM-IDS involves simple LoRA Fine-Tuning rather than Full Fine-Tuning, which explains the slightly lower accuracy compared to BT-TPF. Full Fine-Tuning refers to optimizing all the parameters of a model. This method can maximize the performance of the model. LoRA Fine-Tuning freezes the original parameters of the LLM and only retrains some parameters using the fine-tuning data, simplifying the training process. Using LoRA Fine-Tuning may sacrifice a certain portion of the model quality, but it has a higher training speed and uses lower memory. The result of Table 4 shows that LLMs can perform at a high level with fine-tuning, suggesting a promising approach for intrusion detection, laying a solid foundation for the subsequent application of the RLFE-IDS framework.

Finally, the comparison of RLFE-IDS with four representative few-shot learning models is made, the results are shown in Table 6, further emphasizes the effectiveness and superiority of RLFE-IDS in malicious network traffic classification under few-shot learning conditions. In this experiment, the intrusion detection task is treated as a 5-shot few-shot learning problem, where the model is provided with only five samples per new class to classify network traffic. According to the experimental results in the table, FC-Net achieves an accuracy of 94.13%, and FS-IDS reaches 97.51%. MAML has an accuracy of 92.19%, while BFS-NID achieves an accuracy of 93.17%, slightly better than MAML but still lower than FS-IDS. RLFE-IDS, however, outperforms all these approaches with an impressive accuracy of 98.02%. The results suggest that RLFE-IDS has strong generalization ability in the 5-shot task, enabling it to better adapt to few-shot learning scenarios. This is particularly evident in intrusion detection tasks, where it significantly improves classification accuracy.

4.5. Experimental analysis

Extensive experiments are conducted to demonstrate the effectiveness and robustness of RLFE-IDS. Figs. 6 and 7 show the changes in loss and accuracy during the training of FE-Net. Table 7 presents the accuracy of RLFE-IDS on various datasets. Table 8 presents the recall of RLFE-IDS on various datasets. Figs. 8, 9, and 10 display the confusion matrices for the results of each dataset in the 1-shot and 5-shot tasks.

From Figs. 6 and 7, it is observed that after 100 epochs of training, both the loss and accuracy stabilize. The accuracy fluctuates by approximately 2%, reaching a peak of 97.55%. This suggests that the vectors

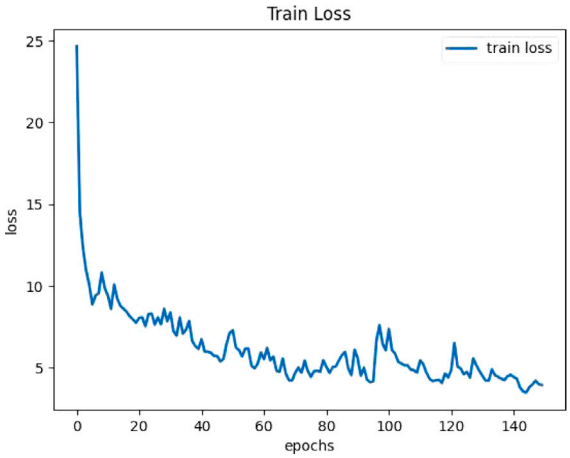


Fig. 6. The train loss curve of FE-Net.

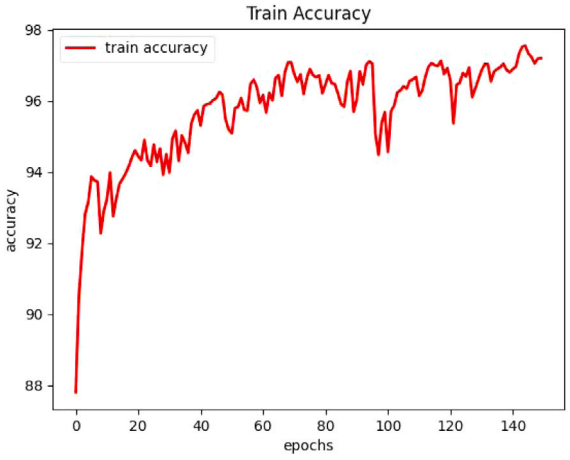


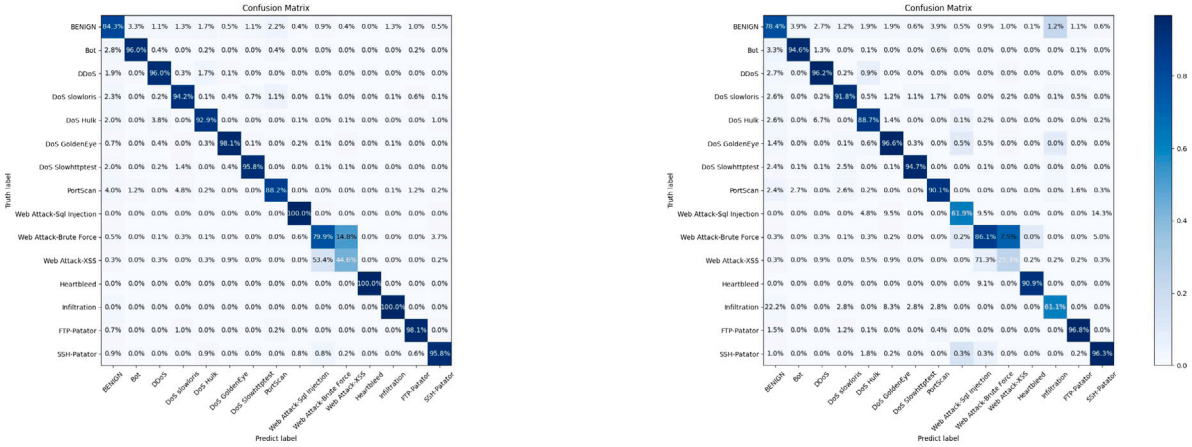
Fig. 7. The train accuracy curve of FE-Net.

embedded by the FE-Net model effectively preserve the correlation between features and labels, mitigating the differences between text and network traffic data, and ensuring reliable results for subsequent retrievals.

The results in Table 7 demonstrate that RLFE-IDS exhibits strong generalization ability and robustness. Notably, although the test set is changed, FE-Net is not retrained. The training set used throughout is CIC-IDS2017, and RLFE-IDS only replaces the data in the knowledge database for different dataset. In other words, apart from the attack data in CIC-IDS2017, the attack patterns in other datasets were either unknown attacks or known attack variants to RLFE-IDS. RLFE-IDS could only access limited information about these attack patterns from the knowledge database and conduct detection based on this sparse information. Additionally, whenever new data is detected, RLFE-IDS stores its information, feature vector, and label into the knowledge database to enable continuous knowledge updates.

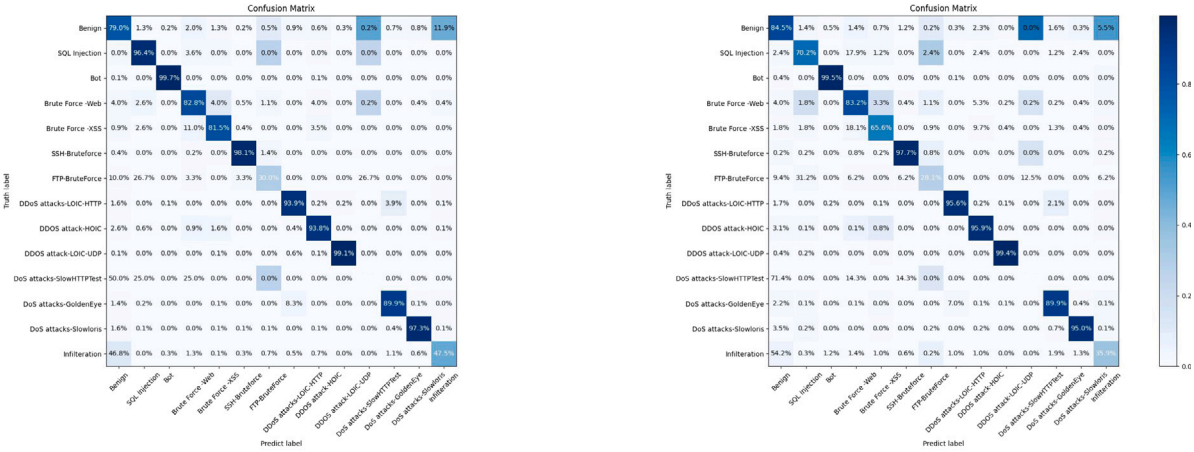
From Table 7, it is observed that in the 1-shot task, the accuracy for CIC-IDS2017, CSE-CIC-IDS2018, and CIC-DDoS2019 are 98.56%, 93.68%, and 99.00%, respectively. In the 3-shot task, the accuracy for CIC-IDS2017, CSE-CIC-IDS2018, and CIC-DDoS2019 are 98.45%, 93.25%, and 98.81%, respectively. In the 5-shot task, the accuracy for CIC-IDS2017, CSE-CIC-IDS2018, and CIC-DDoS2019 are 98.02%, 92.42%, and 98.87%, respectively.

Table 7 shows the accuracy of RLFE-Net. The accuracy of CIC-IDS2017 was consistently higher than that of CSE-CIC-IDS2018 across all tasks. This is because RLFE-IDS was trained solely on CIC-IDS2017,



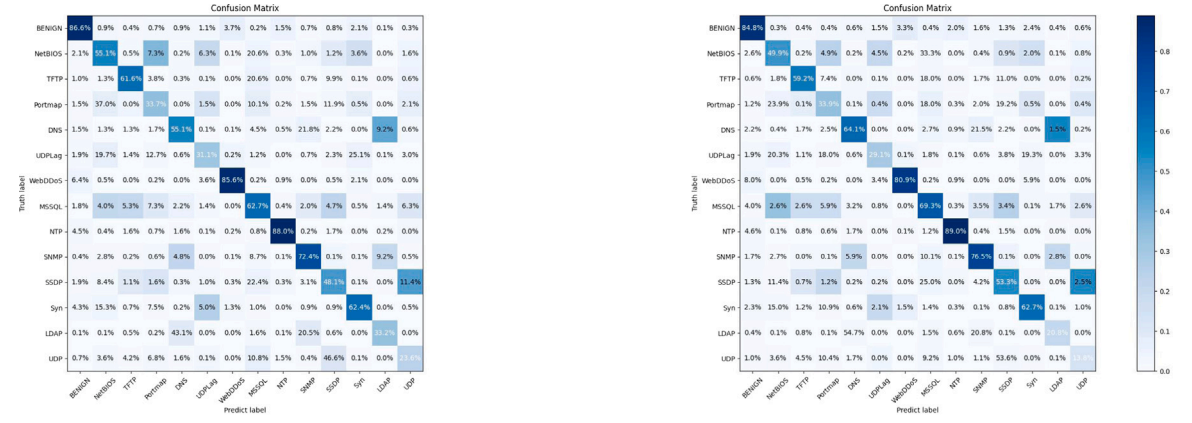
1-shot 5-shot

Fig. 8. The confusion matrices of CIC-IDS2017 in the 1-shot and 5-shot task.



1-shot 5-shot

Fig. 9. The confusion matrices of CSE-CIC-IDS2018 in the 1-shot and 5-shot task.



1-shot 5-shot

Fig. 10. The confusion matrices of CIC-DDoS2019 in the 1-shot and 5-shot task.

Table 7
The Accuracy of three datasets in the 1-shot task and 5-shot task.

Datasets	Accuracy		
	1-shot	3-shot	5-shot
CIC-IDS2017	98.56%	98.45%	98.02%
CSE-CIC-IDS2018	93.68%	93.25%	92.42%
CIC-DDoS2019	99.00%	98.81%	98.87%
CICIoT2023	98.59%	98.69%	98.81%
Real Environment	98.25%	98.34%	98.58%

Table 8
The Recall of three datasets in the 1-shot task and 5-shot task.

Datasets	Recall		
	1-shot	3-shot	5-shot
CIC-IDS2017	99.79%	99.68%	99.23%
CSE-CIC-IDS2018	98.46%	98.56%	98.12%
CIC-DDoS2019	99.01%	99.88%	98.68%
CICIoT2023	99.33%	99.37%	99.11%
Real Environment	98.23%	98.45%	98.76%

while CSE-CIC-IDS2018 includes attack data not present in CIC-IDS2017. As a result, the model is less familiar with this data, leading to a slight decrease in accuracy about 5%.

However, some results in Table 7 are surprising. For example, the accuracy of CIC-IDS2017 is lower than that of CIC-DDoS2019 in all tasks. This may be due to the nature of the datasets: the attack data in CIC-DDoS2019 are all DDoS-related, whereas CIC-IDS2017 includes a wider range of attack types. The higher correlation among the DDoS attack data in CIC-DDoS2019 may explain why, even with some incorrect examples, the model is able to classify more accurately.

Additionally, an unusual trend is observed in the 1-shot, 3-shot, and 5-shot tasks. The 1-shot task achieves the highest accuracy, followed by the 3-shot task, with the 5-shot task showing the lowest accuracy. This could be due to the need for more sample references in the 3-shot and 5-shot tasks, which may introduce incorrect samples that impact the model's judgment, leading to a decline in classification accuracy. To address this, fine-tuning of the model may be required.

RLFE-IDS also performs well on the CICIoT2023 dataset. The accuracies of the 1-shot, 3-shot, and 5-shot tasks are 98.59%, 98.69%, and 98.81% respectively, showing an increasing trend in sequence, which is in line with the normal situation. In addition, although the data sources in the CICIoT2023 dataset are different from those in the other three datasets, the accuracy of RLFE-IDS on it exceeds that on the CIC-IDS2017 dataset, further demonstrating the strong generalization ability of RLFE-IDS.

Table 8 shows the recall of RLFE-Net. It is shown that the recall of four datasets are all more than 98%, which demonstrates the excellent detection and capture capabilities for attack data of RLFE-IDS. Although the recall has a delight decrease in, this may be attributed to the dataset containing more unknown attacks and the altered attack patterns for RLFE-IDS.

At last, the 1-shot, 3-shot, and 5-shot tasks were executed respectively in the real network environment, and each task ran for two days with RLFE-IDS. The accuracy of RLFE-IDS in the real network environment is presented in Table 7, and the recall is presented in Table 8. The accuracies of the 1-shot, 3-shot, and 5-shot tasks are 98.25%, 98.34%, and 98.58% respectively, and the recall of the 1-shot, 3-shot, and 5-shot tasks are 98.23%, 98.45%, and 98.76% respectively, when handling real-world network traffic data, demonstrating strong adaptability to real network environment. In particular, its average processing time of the 1-shot, 3-shot, and 5-shot tasks ranges from 10 to 15 s for 10Gbps throughput, indicating acceptable latency.

Figs. 8, 9, and 10 show the confusion matrices for CIC-IDS2017, CSE-CIC-IDS2018, and CIC-DDoS2019 in the 1-shot and 5-shot tasks, respectively. By comparing these three figures, it is clear that the model performs best on CIC-IDS2017. Observation shows that as the number of training samples increased from 1 to 5, the diagonal intensity of confusion matrices for all three datasets significantly increased, indicating improved correct classification capability. Meanwhile, non-diagonal regions exhibited reduced inter-class confusion, with CIC-DDoS2019 notably demonstrating sharper discrimination of DDoS variants under 5-shot conditions. Notably, false positive rates are below 5% for all attack categories except Infiltration, FTP-BruteForce, and DoS attacks-SlowHTTPTest, which exceeded 10%. On CIC-IDS2017 and CSE-CIC-IDS2018, classification accuracy exceedes 90% for all but one or two data categories. While accuracy droppes significantly on CIC-DDoS2019, recall rates remaines stable due to the dataset's focus on DDoS attacks—many of which are not present in CIC-IDS2017. On CSE-CIC-IDS2018 and CIC-DDoS2019, although several attack categories exhibit low accuracy, the misclassified categories are highly correlated with their actual categories. This situation, where the categories are difficult to distinguish (like oranges and tangerines), highlights an area for future improvement. It suggests that enhancing the model's ability to distinguish between similar categories should be a focus for future development.

Table 9
The ablation experiment result of RAG and FE-Net.

LLM	RAG	FE-Net	Accuracy
✓	✗	✗	26.83%
✓	✓	✗	82.52%
✓	✓	✓	98.02%

4.6. Ablation experiment

In the ablation experiments, two main types of experiments are conducted: one focuses on calculating cost, and the other on calculating accuracy.

In the accuracy experiment, an ablation comparison between the RAG framework and FE-Net is performed. In the experiment without the RAG framework, this article directly uses the processed network data to query the LLM without fine-tuning and let it make classification decisions. In the experiment without FE-Net, the open-source embedding model called "bge-english-large-v1.5" [45] is utilizes, which is also developed by Zhipu AI and has demonstrated excellent performance in the NLP field. The results of these experiments are shown in Table 9.

It was found that the LLM without the RAG framework struggles to determine whether the network data represented attack data, resulting in low accuracy. In some cases, the model even produces responses such as "I can't determine whether this data is attack data" or "Insufficient conditions, unable to judge". This indicates that without fine-tuning, a standalone LLM is not suitable for intrusion detection, as it lacks the domain-specific knowledge required and cannot learn from the data. However, the RAG framework provides a knowledge database for the LLM, enabling it to learn from the provided knowledge and generate the desired responses.

Experiments using open-source embedding models also have a lower accuracy rate. Regarding the use of the open-source embedding model, the vectors generated by this model are more general and lack specificity. As a result, during the retrieval process, it is unable to access the most relevant and accurate knowledge. This can lead to the LLM being provided with incorrect information, causing classification errors and ultimately resulting in lower accuracy in the final classification.

The cost comparison experiment mainly focused on the differences in cost between LLM-IDS and RLFE-IDS, considering two aspects: time and GPU memory. The results are shown in Table 10. In terms of time, training time and testing time of the two models are evaluated. Training time represents the learning efficiency of the model, while testing time reflects its deployment efficiency. For LLM-IDS, the training time refers

Table 10
The comparison in terms of Time Consuming and Memory for LLM-IDS and RLFE-IDS.

Models	Accuracy	Recall	Training Time	Testing Time	GPU Memory
LLM-IDS (Fine-tuning)	99.36%	99.67%	60 h	16 h	322,918 MB
RLFE-IDS (Ours)	98.56%	99.79%	10 h	6 h	14,962 MB

to the fine-tuning time, and the testing time refers to the time taken for testing the LLM. For RLFE-IDS, the training time refers to the time taken to train FE-Net, and the testing time refers to the time taken to test RLFE-IDS.

Although LLM-IDS achieves slightly higher accuracy, RLFE-IDS outperforms in terms of recall rate, training time, testing time and GPU memory usage, significantly improving efficiency and resource utilization. In addition, LLM-IDS utilized a total of 8 GPUs. Each GPU consumed approximately 40,960 MB of memory, resulting in a total GPU memory usage of 322,918 MB. But RLFE-IDS only uses 1 GPU, a total GPU memory usage of 14,962 MB. Therefore, RLFE-IDS not only performs comparably to the LLM but also offers significant advantages in computational resource consumption and processing speed.

5. Conclusion

This article first explored the feasibility of using a LLM with strong generalization capabilities for intrusion detection. By fine-tuning the LLM, LLM is successfully applied to IDS called LLM-IDS. This article then proposed RLFE-IDS, to overcome the difficulties in deploying LLM and the differences between text data and network data. In RLFE-IDS, this article introduced the RAG framework and developed FE-Net, an embedding model tailored for intrusion detection. RAG provided a knowledge database for LLM, enabling it to perform knowledge retrieval using vectors. This allowed LLM to learn from other domains without requiring fine-tuning. Meanwhile, FE-Net embedded network data, ensuring that the output vectors preserve the correlation between features and labels, thereby improving the accuracy of retrieval results. As demonstrated on four commonly datasets, RLFE-IDS not only outperformed existing solutions in terms of performance and robustness but also significantly reduced resource consumption.

However, the performance of RLFE-IDS on other test sets was not satisfactory. First, the generation of RLFE-IDS is need to improve, because the performance of RLFE-DIS decreases in datasets other than the dataset it was trained on. Furthermore, due to the prompt of various LLMs have word count limitations and are charged according to tokens, it cannot provide a lot of high-quality samples to the LLMs, which may lead to LLMs generating incorrect answer. On the other hand, RLFE-IDS is trained and tested on controlled datasets, but real-world performance may differ due to variations in traffic volume, encryption, and obfuscation techniques.

In the future, firstly, it is important to improve the generation ability of RLFE-IDS, during the training process, a joint training algorithm can be used to jointly train multiple training sets. Then, it will continue to explore ways to enhance the ability of the embedding model, improve the retrieval strategy, and optimize the quality of embedding and retrieval. It will be committed to exploring how to use as few prompt words as possible to provide higher-quality samples to LLMs, enabling them to conduct more accurate classifications. Besides, it needs to check the quality of the queried data. It is possible to incorporate a manual feedback mechanism, where experts will conduct the inspection and then provide the results as feedback to the model. Finally, in the face of the challenges in the real world, it will incorporate a hierarchical detection mechanism. For data from different layers, it will use the corresponding algorithms to detect. For example, deep learning algorithms can be used to encrypt and decrypt in the data layer, and RLFE-IDS can be used in the network layer.

CRediT authorship contribution statement

Xuewei Li: Writing – review & editing, Methodology, Formal analysis. **Zengyang Zheng:** Writing – original draft, Validation, Methodology, Investigation. **Mankun Zhao:** Writing – review & editing. **Yue Zhao:** Writing – review & editing. **Lifeng Shi:** Data curation. **Baoliang Wang:** Writing – review & editing, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Key Research and Development Program of China (No. 2024YFE0115700) and National Natural Science Foundation of China (No. 62402335).

Data availability

Data will be made available on request.

References

[1] H.-J. Liao, C.-H.R. Lin, Y.-C. Lin, et al., Intrusion detection system: A comprehensive review, J. Netw. Comput. Appl. 36 (1) (2013) 16–24.
[2] R.K. Panchal, R. Snehkunj, V.V. Panchal, A survey on network-based intrusion detection system using learning techniques, in: 2024 5th International Conference on Image Processing and Capsule Networks, ICIPCN, IEEE, 2024, pp. 740–747.
[3] A. Aldweesh, A. Derhab, A.Z. Emam, Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues, Knowl.-Based Syst. 189 (2020) 105124.
[4] A.N. Özalp, Z. Albayrak, M. Çakmak, E. ÖzdoĞan, Layer-based examination of cyber-attacks in IoT, in: 2022 International Congress on Human-Computer Interaction, Optimization and Robotic Applications, HORA, IEEE, 2022, pp. 1–10.
[5] X. Li, M. Zhu, L.T. Yang, et al., Sustainable ensemble learning driving intrusion detection model, IEEE Trans. Dependable Secur. Comput. 18 (4) (2021) 1591–1604.
[6] J. Lan, X. Liu, B. Li, et al., MEMBER: A multi-task learning model with hybrid deep features for network intrusion detection, Comput. Secur. 123 (2022) 102919.
[7] A. Diro, N. Chilamkurti, V.-D. Nguyen, W. Heyne, A comprehensive study of anomaly detection schemes in IoT networks using machine learning algorithms, Sensors 21 (24) (2021) 8320.
[8] C. Xu, J. Shen, X. Du, A method of few-shot network intrusion detection based on meta-learning framework, IEEE Trans. Inf. Forensics Secur. 15 (2020) 3540–3552.
[9] D. Anisuzzaman, J.G. Malins, P.A. Friedman, et al., Fine-tuning llms for specialized use cases, Mayo Clin. Proc. Digit. Heal. (2024).
[10] M. Shanahan, Talking about large language models, Commun. ACM 67 (2) (2024) 68–79.
[11] J. Allen, M. Swift, W. de Beaumont, Deep semantic analysis of text, in: Semantics in Text Processing. STEP 2008 Conference Proceedings, 2008, pp. 343–354.
[12] D. Apiletti, E. Baralis, T. Cerquitelli, V. D'Elia, Characterizing network traffic by means of the NetMine framework, Comput. Netw. 53 (6) (2009) 774–789.
[13] E.C. Uwazie, A.A. Obiniyi, M. Olalere, et al., Comparison of random forest, K-nearest neighbor, and support vector machine classifiers for intrusion detection system, in: 2024 International Conference on Science, Engineering and Business for Driving Sustainable Development Goals, SEB4SDG, IEEE, 2024, pp. 1–6.
[14] C. Fu, Q. Li, M. Shen, et al., Realtime robust malicious traffic detection via frequency domain analysis, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 3431–3446.
[15] A.N. Ozalp, Z. Albayrak, Detecting cyber attacks with high-frequency features using machine learning algorithms, Acta Polytech. Hung. (2022).

[16] T.-T.-H. Le, H. Kim, H. Kang, et al., Classification and explanation for intrusion detection system based on ensemble trees and SHAP method, Sensors 22 (3) (2022) 1154.
[17] V. Chang, S. Boddu, Q.A. Xu, et al., Intrusion detection and prevention with machine learning algorithms, Int. J. Grid Util. Comput. 14 (6) (2023) 617–631.
[18] W. Wang, M. Zhu, X. Zeng, et al., Malware traffic classification using convolutional neural network for representation learning, in: 2017 International Conference on Information Networking, ICOIN, IEEE, 2017, pp. 712–717.
[19] J.-S. Lee, Y.-C. Chen, C.-J. Chew, et al., CoNN-IDS: Intrusion detection system based on collaborative neural networks and agile training, Comput. Secur. 122 (2022) 102908.
[20] L. Nie, Y. Wu, X. Wang, et al., Intrusion detection for secure social internet of things based on collaborative edge computing: a generative adversarial network-based approach, IEEE Trans. Comput. Soc. Syst. 9 (1) (2021) 134–145.
[21] T. Hospedales, A. Antoniou, P. Micaelli, et al., Meta-learning in neural networks: A survey, IEEE Trans. Pattern Anal. Mach. Intell. 44 (9) (2021) 5149–5169.
[22] J. Yang, H. Li, S. Shao, et al., FS-IDS: A framework for intrusion detection based on few-shot learning, Comput. Secur. 122 (2022) 102899.
[23] C. Lu, X. Wang, A. Yang, et al., A few-shot-based model-agnostic meta-learning for intrusion detection in security of Internet of Things, IEEE Internet Things J. 10 (24) (2023) 21309–21321.
[24] Z. Shi, M. Xing, J. Zhang, et al., Few-shot network intrusion detection based on model-agnostic meta-learning with l2f method, in: 2023 IEEE Wireless Communications and Networking Conference, WCNC, IEEE, 2023, pp. 1–6.
[25] B. Mann, N. Ryder, M. Subbiah, et al., Language models are few-shot learners, 2020, arXiv preprint arXiv:2005.141651.
[26] A. Roberts, C. Raffel, K. Lee, et al., Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer, Tech. Rep., Google, 2019.
[27] A. Radford, K. Narasimhan, T. Salimans, et al., Improving language understanding by generative pre-training, 2018.
[28] A. Radford, J. Wu, R. Child, et al., Language models are unsupervised multitask learners, OpenAI Blog 1 (8) (2019) 9.
[29] W.X. Zhao, K. Zhou, J. Li, et al., A survey of large language models, 2023, arXiv preprint arXiv:2303.18223.
[30] S. Bubeck, V. Chandrasekaran, R. Eldan, et al., Sparks of artificial general intelligence: Early experiments with gpt-4, 2023, arXiv preprint arXiv:2303.12712.
[31] A. Zeng, B. Xu, B. Wang, et al., ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools, 2024, arXiv:2406.12793.
[32] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, et al., Characterization of tor traffic using time based features, in: International Conference on Information Systems Security and Privacy, vol. 2, SciTePress, 2017, pp. 253–262.
[33] B.H. Ali, N. Sulaiman, S. Al-Haddad, et al., Ddos detection using active and idle features of revised cicflowmeter and statistical approaches, in: 2022 4th International Conference on Advanced Science and Engineering, ICOASE, IEEE, 2022, pp. 148–153.
[34] Y. Han, C. Liu, P. Wang, A comprehensive survey on vector database: Storage and retrieval technique, challenge, 2023, arXiv preprint arXiv:2310.11703.
[35] J. Snell, K. Swersky, R. Zemel, Prototypical networks for few-shot learning, Adv. Neural Inf. Process. Syst. 30 (2017).
[36] W. Fan, Y. Ding, L. Ning, et al., A survey on rag meeting llms: Towards retrieval-augmented large language models, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 6491–6501.
[37] Q. Dong, L. Li, D. Dai, et al., A survey on in-context learning, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 1107–1128.
[38] A.H. Azizan, S.A. Mostafa, A. Mustapha, C.F.M. Foozy, M.H.A. Wahab, M.A. Mohammed, B.A. Khalaf, A machine learning approach for improving the performance of network intrusion detection systems, Ann. Emerg. Technol. Comput. (AETiC) 5 (5) (2021) 201–208.
[39] W. Qiu, Y. Ma, X. Chen, et al., Hybrid intrusion detection system based on Dempster-Shafer evidence theory, Comput. Secur. 117 (2022) 102709.
[40] H. Chen, Z. Wang, S. Yang, et al., Intrusion detection using synaptic intelligent convolutional neural networks for dynamic internet of things environments, Alex. Eng. J. 111 (2025) 78–91.
[41] Z. Li, W. Yao, A two stage lightweight approach for intrusion detection in Internet of Things, Expert Syst. Appl. 257 (2024) 124965.
[42] Z. Wang, J. Li, S. Yang, et al., A lightweight IoT intrusion detection model based on improved BERT-of-theseus, Expert Syst. Appl. 238 (2024) 122045.
[43] A. Dosovitskiy, An image is worth 16x16 words: Transformers for image recognition at scale, 2020, arXiv preprint arXiv:2010.11929.
[44] L. Du, Z. Gu, Y. Wang, et al., A few-shot class-incremental learning method for network intrusion detection, IEEE Trans. Netw. Serv. Manag. (2023).
[45] S. Xiao, Z. Liu, P. Zhang, et al., C-pack: Packaged resources to advance general Chinese embedding, 2023, arXiv:2309.07597.